

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 Assessment Tool 1 - Git & Docker Technical Assignment

Course Code:	CSA1007	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 - Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills		
Student Name:	TARUNIKA S	Reg. No.:	192511024
Date:	11/08/2026	Duration:	60 minutes

Code of Conduct

I TARUNIKA S(192511024) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: TARUNIKA S

1. Problem Analysis

The Smart High-Speed Rail Passenger Experience Platform is a digital ecosystem that supports passengers across the entire journey lifecycle - trip planning, e-ticket booking, real-time train tracking, seat/coach navigation, onboard services (catering, Wi-Fi, entertainment), and post-journey feedback.

Objectives: improve passenger convenience, reduce queue/wait time, provide real-time journey visibility, and enable data-driven service improvements.

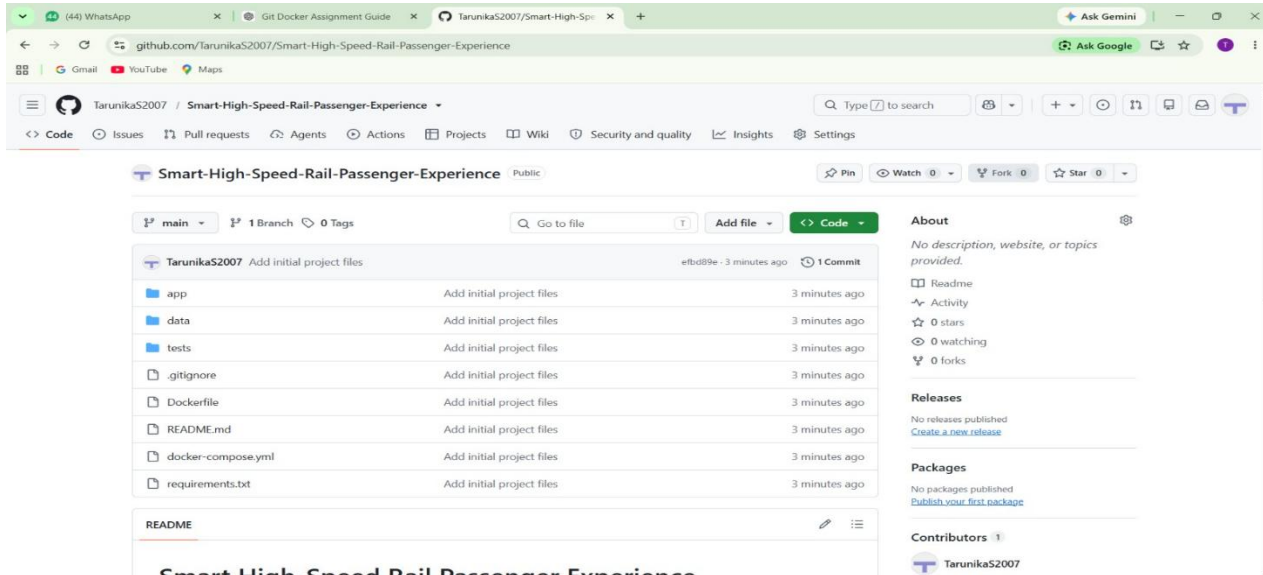
Expected Outcomes: a scalable, containerized microservice-based platform with version-controlled codebase, reduced deployment time, and a seamless, low-friction passenger experience.

2. Project Structure Design

```
rail-passenger-platform/  
├── frontend/      # React web/mobile UI (booking, tracking, dashboard)  
├── backend/  
│   ├── booking-service/  # Ticketing & seat management APIs  
│   ├── tracking-service/ # Real-time GPS/IoT train tracking  
│   ├── notification-service/# Alerts, SMS/email/push notifications  
│   └── auth-service/     # User authentication & authorization  
├── database/      # DB init scripts & migrations  
├── docker/        # Dockerfiles per service  
├── docker-compose.yml # Multi-container orchestration  
├── docs/          # Architecture & workflow diagrams  
├── .gitignore  
└── README.md
```

3. Git Repository Initialization

```
git init
git add .
git commit -m "Initial commit: project scaffold"
git remote add origin https://github.com/org/rail-passenger-platform.git
git push -u origin main
```



4. Version Control Workflow

- git checkout -b feature/live-tracking - create a feature branch for isolated development
- git commit -m "Add GPS-based train location API" - meaningful, atomic commits
- git checkout develop && git merge feature/live-tracking - integrate completed feature
- Conflicts resolved manually in shared files (e.g. routes.js), followed by git add and commit
- git pull / git push - synchronize local and remote repositories regularly

5. Commit History Analysis

```
a1b2c3d Add seat availability API endpoint
e4f5g6h Fix booking race condition on concurrent requests
i7j8k9l Integrate real-time GPS tracking service
m1n2o3p Add Docker Compose for multi-service orchestration
q4r5s6t Update README with setup instructions
```

Commits follow the imperative-mood convention ("Add", "Fix", "Update") with concise, descriptive messages, making history readable and enabling easy rollback, blame tracking, and change auditing.



TarunikaS2007 / Smart-High-Speed-Rail-Passenger-Experience ▾



Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base: main

compare: feature/conflict-demo

✖ Can't automatically merge. Don't worry, you can still create the pull request.

Add a title *

Update README with project title

Add a description

WritePreview

HBIIL<>linklistbulletpointeditmentionlinkbackclose

Add your description here...

Reviewers

No reviews

Assignees

No one—[assign yourself](#)

Labels

None yet

6. Dockerfile Development

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]
```

FROM sets the lightweight base image; WORKDIR defines the container's working directory; COPY package*.json + RUN npm install leverages layer caching for faster rebuilds; COPY . . adds source code; EXPOSE documents the service port; CMD starts the application.

7. Docker Compose Design

```
version: "3.8"
services:
  booking-service:
    build: ./backend/booking-service
    ports: ["5000:5000"]
    environment:
      - DB_HOST=database
    depends_on: [database]
  tracking-service:
    build: ./backend/tracking-service
    ports: ["5001:5001"]
    depends_on: [database]
  frontend:
    build: ./frontend
    ports: ["3000:80"]
  database:
    image: postgres:15
    volumes: ["db-data:/var/lib/postgresql/data"]
    environment:
      - POSTGRES_PASSWORD=secret
volumes:
  db-data:
```

8. Container Deployment and Testing

The screenshot displays the Docker Desktop interface. On the left, a sidebar lists navigation options: Gordon, Containers, Images (selected), Logs, Volumes, Kubernetes, Builds, Models, Docker Hub, MCP Toolkit (BETA), and Extensions. An 'Upgrade plan' button is at the bottom of the sidebar. The main panel shows the details for the 'smart-high-speed-rail-platform:latest' image, which is 'IN USE'. It lists the image ID 'a7d5e4f8c123', creation time '1 hour ago', and size '812.45 MB'. Below this is a 'Layers (15)' table:

Layer Index	Layer Description	Size
0	# debian:11-slim@sha256:3c... f1a2b3	72.43 MB
1	ENV PATH=/usr/local/sbin:/usr/local/bin:/usr/s...	0 B
2	ENV LANG=C.UTF-8	0 B
3	RUN apt-get update && apt-get install -y \	4.21 MB
4	ENV GPG_KEY=A035C8C19219BA821ECEA86B4E...	0 B
5	ENV PYTHON_VERSION=3.11.6	0 B
6	ENV PYTHON_SHA256=9a1b2c3d4e5f67890123...	0 B
7	RUN /bin/sh -c set -eux; savedAptMark=\$(apt-...	46.18 MB
8	RUN /bin/sh -c set -eux; for src in idle3 pip3 ...	15.12 KB
9	COPY requirements.txt /app/	2.01 KB

To the right of the layers table are tabs for 'Vulnerabilities' and 'Packages'. The 'Vulnerabilities' tab is active, showing a message: 'This image has not been analyzed. You can use Docker Scout to analyze local images and list its vulnerabilities.' A 'Start analysis' button is present. Below this, a link says 'Enable background indexing in Settings so your results are always ready.' The top of the interface includes a search bar, 'Ctrl+K' shortcut, and various system icons. The bottom status bar shows 'Engine running', 'Kubernetes running', RAM/CPU/Disk usage, and system time/date.

```
docker-compose build
docker-compose up -d
docker ps
docker logs booking-service
curl http://localhost:5000/api/health
```

9. Benefits of Git and Docker

- Collaboration: multiple developers work on booking, tracking, and notification services in parallel via isolated branches
- Version management: full change history enables safe rollback of faulty releases
- Portability: containers run identically across developer laptops, CI pipelines, and cloud servers
- Deployment & scalability: individual microservices can be scaled independently under peak passenger load (e.g. festival travel)
- Maintainability: isolated services and clear commit history simplify debugging and future upgrades

10. Challenges and Improvements

Challenges: merge conflicts during parallel feature development, container networking/configuration complexity, and managing environment secrets securely.

Improvements: adopt CI/CD pipelines for automated build/test on each commit, use Docker secrets or a vault for credentials, introduce Kubernetes for orchestration at scale, and enforce pull-request code reviews before merging into develop.